

Schema de Semnătură Digitală Bazată pe Identitate Guillou-Quisquater (în Z_n): Implementare și Analiză Comparativă

Flavia Martinecz

Universitatea Politehnica Timișoara
Facultatea de Automatica și Calculatoare
Master SISC - Tehnici Criptografice Moderne
An I, Semestrul I

Abstract—Această lucrare prezintă implementarea și analiza comparativă a schemei de semnătură digitală bazată pe identitate Guillou-Quisquater (GQ), care operează în grupul multiplicativ Z_n^* . Schema GQ oferă o alternativă eficientă la metodele tradiționale de semnătură digitală prin eliminarea necesității certificatelor de cheie publică, folosind direct identitatea utilizatorului ca cheie publică.

Toate operațiile criptografice (exponențiere, înmulțire, inversare) sunt efectuate modulo n , unde $n = p \times q$ este produsul a două numere prime mari. Implementarea a fost realizată în Java și testată comparativ cu algoritmul RSA standard. Rezultatele experimentale demonstrează că schema GQ oferă performanțe competitive în ceea ce privește timpul de semnare și verificare, menținând în același timp un nivel ridicat de securitate bazat pe dificultatea problemei RSA în Z_n^* .

Keywords—semnătură digitală, identitate, Guillou-Quisquater, grup multiplicativ Z_n^* , RSA, criptografie cu cheie publică

I. INTRODUCERE

Criptografia bazată pe identitate (Identity-Based Cryptography - IBC) reprezintă o paradigmă importantă în domeniul securității informatice, permițând utilizarea informațiilor publice de identificare (precum adrese de email) direct ca și chei publice. Schema Guillou-Quisquater, propusă în 1988, este unul dintre primele și cele mai influente protocoale de acest tip.

Spre deosebire de infrastructura tradițională de cheie publică (PKI), unde fiecare utilizator necesită un certificat digital emis de o autoritate de certificare, în sistemele bazate pe identitate, cheia publică a utilizatorului este derivată direct din identitatea sa. Această abordare simplifică semnificativ managementul cheilor și reduce costurile asociate cu infrastructura PKI.

Obiectivele principale ale acestei lucrări sunt: (1) implementarea completă a schemei GQ în Java, (2) validarea corectitudinii implementării prin teste funcționale, și (3) evaluarea comparativă a performanțelor față de algoritmul RSA standard.

II. FUNDAMENTELE TEORETICE

A. Grupul Multiplicativ Z_n^*

Schema Guillou-Quisquater operează în grupul multiplicativ Z_n^* , unde $n = p \times q$ este produsul a două numere prime mari. Pentru a defini acest grup, introducem mai întâi notația $\gcd(a, n)$ care reprezintă cel mai mare divizor comun (greatest common divisor) al numerelor a și n , adică cel mai mare număr întreg pozitiv care divide simultan atât pe a cât și pe n .

Grupul Z_n^* este definit ca mulțimea tuturor numerelor întregi din intervalul $[1, n-1]$ care sunt coprime cu n (adică nu au factori comuni cu n în afară de 1):

$$Z_n^* = \{a \in Z_n \mid \gcd(a, n) = 1\} \quad (1)$$

De exemplu, dacă $n = 15$, atunci $Z_{15}^* = \{1, 2, 4, 7, 8, 11, 13, 14\}$, deoarece acestea sunt singurele numere din $[1, 14]$ care nu au factori comuni cu 15 (excluzând multiplii de 3 și 5). Pentru $n = p \times q$ cu p, q prime, condiția $\gcd(a, n) = 1$ exclude doar multiplii lui p și ai lui q .

Acest grup este închis față de operația de înmulțire modulară și are ordinul $\phi(n) = (p-1)(q-1)$, unde ϕ este funcția Euler. Fiecare element din Z_n^* are un invers multiplicativ unic modulo n . Structura grupului Z_n^* este fundamentală pentru criptografia RSA și schemele derivate, inclusiv GQ.

Proprietățile esențiale ale grupului Z_n^* pentru schema GQ sunt: (1) operația de exponențiere modulară $a^k \bmod n$ este eficient calculabilă, (2) extragerea rădăcinii de ordinul v (calcularea $a^{(1/v)} \bmod n$) este computațional dificilă fără cunoașterea factorizării lui n , și (3) inversul modular poate fi calculat eficient.

B. Problema Factorizării și Problema RSA

Securitatea schemei GQ se bazează pe dificultatea computațională a problemei RSA, care este strâns legată de problema factorizării. Problema RSA constă în calcularea rădăcinii de ordinul v modulo n , adică găsirea lui x astfel încât $x^v \equiv c \pmod{n}$, fără a cunoaște factorizarea lui n .

Fiind dat un număr $n = p \times q$, unde p și q sunt numere prime mari (de exemplu, 512 sau 1024 biți fiecare), determinarea factorilor p și q este considerată intractabilă. Cel mai eficient algoritm cunoscut pentru factorizare este Number Field Sieve (NFS) cu complexitate subexponențială $O(\exp(c \cdot (\ln n)^{1/3}) \cdot (\ln \ln n)^{(2/3)))$.

III. SCHEMA GUILLOU-QUISQUATER ÎN Z_n^*

A. Configurarea Sistemului

Autoritatea de Certificare (CA) generează parametrii publici ai sistemului care definesc grupul Z_n^* în care se vor efectua toate calculele. CA alege doi factori primi mari p și q de dimensiuni egale (de exemplu, 512 biți fiecare pentru n de 1024 biți), calculează modulul $n = p \times q$ și funcția Euler $\phi(n) = (p-1)(q-1)$ care reprezintă ordinul grupului Z_n^* .

Exponentul public v este ales ca fiind 65537 ($= 2^{16} + 1$), valoare standard în criptografia RSA, aleasă pentru eficiența exponențierii (are doar 2 biți de 1 în reprezentarea binară). Se verifică că $\gcd(v, \phi(n)) = 1$ pentru a asigura existența inversului. Exponentul secret s este calculat ca inversul modular în grupul $(Z_{\phi(n)}, \times)$:

$$s = v^{-1} \bmod \phi(n), \text{ astfel încât } v \times s \equiv 1 \pmod{\phi(n)} \quad (2)$$

Parametrii publici (n, v, k) sunt distribuiți tuturor participanților, unde k reprezintă parametrul de securitate (dimensiunea challenge-

ului în biți, tipic 128 sau 256). Parametrii secreți (p, q, s) sunt păstrați exclusiv de CA.

B. Generarea Certificatului în Z^*n

Pentru fiecare utilizator cu identitatea ID (de exemplu, o adresă de email), CA calculează certificatul secret J care este un element din Z^*n . Mai întâi, identitatea este transformată într-un element al grupului prin funcția hash:

$$I = H(ID) \bmod n, \text{ unde } I \in Z^*n \quad (3)$$

Apoi, CA calculează certificatul J ca rădăcina de ordinul v a lui I în Z^*n , folosind exponentul secret s:

$$J = I^s \bmod n, \text{ astfel încât } J^v \equiv I \pmod{n} \quad (4)$$

Certificatul J este transmis în mod sigur utilizatorului și reprezintă secretul său pe care îl va folosi pentru semnarea mesajelor. Relația $J^v \equiv I \pmod{n}$ este esențială pentru verificare și rezultă din teorema lui Euler: $J^v = I^{sv} = I^{(1 + k\phi(n))} = I \times (I^{\phi(n)})^k \equiv I \times 1^k \equiv I \pmod{n}$.

C. Procesul de Semnare în Z^*n

Pentru a semna un mesaj M, utilizatorul efectuează următorii pași, toate operațiile fiind efectuate în grupul Z^*n :

- 1) Alege un număr aleatoriu r din Z^*n :
 $r \leftarrow \{x \in [1, n-1] \mid \gcd(x, n) = 1\}$
- 2) Calculează commitment-ul prin exponențiere în Z^*n :
 $T = r^v \bmod n$
- 3) Calculează challenge-ul folosind funcția hash:
 $d = H(M \parallel T) \bmod 2^k$
- 4) Calculează răspunsul prin înmulțire în Z^*n :
 $y = r \times J^d \bmod n$

Semnătura este tripletul (d, y, ID) unde d este challenge-ul de k biți, $y \in Z^*n$ este răspunsul, iar ID este identitatea semnatarului.

D. Verificarea Semnăturii în Z^*n

Verificatorul validează semnătura efectuând calcule în același grup Z^*n , folosind doar parametrii publici (n, v) și identitatea semnatarului:

- 1) Recalculează elementul identității în Z^*n :
 $I = H(ID) \bmod n$
- 2) Calculează T' , unde $I^{(-d)}$ este inversul lui I^d în Z^*n :
 $T' = y^v \times I^{(-d)} \bmod n$
- 2) Recalculează challenge-ul:
 $d' = H(M \parallel T')$
- 4) Acceptă semnătura dacă și numai dacă $d' = d$

Corectitudinea verificării: Pentru o semnătură validă, avem $y = r \times J^d$, deci $y^v = r^v \times J^{vd} = r^v \times I^d$ (deoarece $J^v = I$). Astfel $T' = y^v \times I^{(-d)} = r^v \times I^d \times I^{(-d)} = r^v = T$, și challenge-urile coincid.

IV. IMPLEMENTARE JAVA

A. Reprezentarea Elementelor din Z^*n

Implementarea a fost realizată în Java folosind clasa `java.math.BigInteger` pentru reprezentarea elementelor din Z^*n . `BigInteger` permite operații cu numere întregi de dimensiune arbitrară, esențiale pentru modulul n de 1024-2048 biți. Operațiile în grupul Z^*n sunt implementate astfel:

Înmulțirea modulară: `a.multiply(b).mod(n)` pentru $a \times b \bmod n$.

Exponențierea modulară: `a.modPow(e, n)` pentru $a^e \bmod n$, implementată eficient folosind metoda `square-and-multiply`.

Inversul modular: `a.modInverse(n)` pentru $a^{(-1)} \bmod n$, calculat cu algoritmul Euclid extins.

B. Structura Codului

Implementarea constă din trei clase Java complementare care acoperă funcționalitatea completă a schemei GQ:

Clasa `GQSignature` reprezintă nucleul implementării și conține algoritmul principal. Are doi constructori: primul pentru Autoritatea de Certificare (CA), care generează parametrii secreți p, q și calculează n, v, s, $\phi(n)$; al doilea pentru utilizatori obișnuiți, care primește doar parametrii publici (n, v, k). Metodele principale sunt: `generareCertificat(identitate)` pentru CA, `semnare(mesaj, identitate, certificat)` și `verificare(mesaj, semnatura)` pentru utilizatori. Clasa internă `Semnatura` încapsulează tripletul (d, y, identitate).

Clasa `GQDemo` oferă o demonstrație completă a funcționării schemei cu doi utilizatori fictivi (Alice și Bob). Parcurge toate etapele protocolului: setup-ul sistemului de către CA, înregistrarea utilizatorilor și generarea certificatelor, semnarea mesajelor de către Alice, verificarea de către Bob, și teste de securitate pentru detectarea mesajelor modificate și a identităților false. Această clasă validează corectitudinea implementării prin scenarii realiste de utilizare.

Clasa `GQBenchmark` realizează măsurători comparative de performanță între schema GQ și algoritmul RSA standard din Java Security API. Testează ambele scheme pentru dimensiuni de cheie de 1024 și 2048 biți, măsurând timpii pentru: generarea cheilor/setup, generarea certificatelor, semnare și verificare. Fiecare operație este executată de 100 de ori după o fază de încălzire (warmup) a JVM pentru rezultate precise. Se compară și dimensiunile semnăturilor rezultate.

C. Generarea Elementelor Aleatoare din Z^*n

Funcția `generareRandom()` generează un element aleatoriu r din Z^*n folosind `java.security.SecureRandom`. Se generează numere de `bitLength(n)` biți până când se obține unul care satisface: $r < n$ și $\gcd(r, n) = 1$. Condiția $\gcd(r, n) = 1$ asigură că $r \in Z^*n$ și are invers multiplicativ.

D. Funcția Hash și Challenge

Pentru funcția hash H s-a utilizat SHA-256 din `java.security.MessageDigest`. Challenge-ul d este calculat ca $H(M \parallel T) \bmod 2^k$, unde concatenarea se face la nivel de string și rezultatul hash-ului este trunchiat la k biți pentru a limita spațiul de căutare al unui atacator.

V. REZULTATE EXPERIMENTALE

S-au efectuat măsurători comparative între schema GQ și algoritmul RSA pentru dimensiuni de cheie de 1024 și 2048 biți. Fiecare operație a fost executată de 100 de ori, calculându-se media timpilor.

Tabelul I: Rezultate pentru chei de 1024 biți

Operație	GQ (ms)	RSA Sign (ms)	RSA Enc (ms)
Setup/Gen. chei	285	142	142
Gen. certificat	3	-	-
Semnare/Criptare	4	6	1
Verificare/Decriptare	2	1	5
Semnătură (bytes)	160	128	128

Tabelul II: Rezultate pentru chei de 2048 biți

Operație	GQ (ms)	RSA Sign (ms)	RSA Enc (ms)
Setup/Gen. chei	1842	523	523
Gen. certificat	18	-	-
Semnare/Criptare	21	35	3
Verificare/Decriptare	7	2	32
Semnătură (bytes)	320	256	256

A. Analiza Rezultatelor

Din rezultatele experimentale se pot observa următoarele tendințe:

Setup-ul sistemului GQ este mai lent decât generarea cheilor RSA deoarece necesită calculul inversului modular $s = v^{-1} \bmod \phi(n)$, operație suplimentară față de RSA standard.

Semnarea GQ este comparabilă sau mai rapidă decât semnarea RSA pentru chei de 2048 biți. Aceasta se datorează faptului că GQ folosește exponențieri cu exponenți mai mici (d are doar k biți) comparativ cu cheia privată RSA.

Verificarea GQ necesită două exponențieri modulare și o inversare, fiind ușor mai lentă decât verificarea RSA care folosește exponentul public mic (65537).

Dimensiunea semnăturii GQ este mai mare deoarece include atât challenge-ul d cât și răspunsul y, în timp ce RSA produce o semnătură de dimensiunea modulului.

VI. DISCUȚII

A. Avantajele Schemei GQ

Schema GQ oferă mai multe avantaje semnificative. Eliminarea necesității certificatelor de cheie publică simplifică infrastructura și

reduce costurile. Identitatea utilizatorului servește direct ca cheie publică, ceea ce face sistemul mai intuitiv.

B. Dezavantajele și Limitările

Principala limitare este problema key escrow: Autoritatea de Certificare cunoaște secretele tuturor utilizatorilor și poate genera semnături în numele oricui. Revocarea identităților compromise necesită mecanisme suplimentare. De asemenea, dimensiunea semnăturii este mai mare decât la RSA.

C. Securitate

Securitatea schemei GQ se reduce la dificultatea problemei factorizării. Pentru parametri recomandați (n de 2048+ biți, k de 256 biți), schema oferă un nivel de securitate de aproximativ 112 biți, comparabil cu RSA-2048.

VII. CONCLUZII

Această lucrare a prezentat o implementare completă și funcțională a schemei de semnătură digitală Guillou-Quisquater. Rezultatele experimentale demonstrează că schema oferă performanțe competitive față de RSA, cu avantajul semnificativ al eliminării necesității certificatelor de cheie publică.

Implementarea validează corectitudinea prin teste de integritate și autenticitate, demonstrând că mesajele modificate sau semnăturile cu identități false sunt corect respinse.

REFERINȚE

- [1] A. Menezes, P. van Oorschot și S. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.
- [2] M. Bellare și A. Palacio, "GQ and Schnorr Identification Schemes: Proofs of Security against Impersonation under Active and Concurrent Attacks," *Cryptology ePrint Archive*, Report 2002.
- [3] L. C. Guillou și J.-J. Quisquater, "A Practical Zero-Knowledge Protocol Fitted to Security Microprocessor Minimizing Both Transmission and Memory," în *Advances in Cryptology — EUROCRYPT '88*, vol. 330, Lecture Notes in Computer Science, pp. 123-128, Springer-Verlag, 1988.
- [4] M. Choudary Gorantla, R. Gangishetti și A. Saxena, "A Survey on ID-Based Cryptographic Primitives," *Cryptology ePrint Archive*, Report 2005.

Proiectul conține 3 fișiere Java:

1. GQSignature.java - Implementarea principală a algoritmului GQ (generare parametri, certificare, semnare, verificare)

```
import java.math.BigInteger;
import java.security.MessageDigest;
import java.security.SecureRandom;

/**
 * Tema proiect:
 * Schema de semnatura digitala bazata pe identitate Guillou-Quisquater (in Zn)
 *
 * @author Flavia Martinecz
 * @university Universitatea Politehnica Timisoara
 * @course Master SISC - Tehnici criptografice moderne - an I sem. I
 */
public class GQSignature {

    // Parametri PUBLICI - toti ii stiu
    private BigInteger n; // Modul RSA:  $n = p \times q$ 
    private BigInteger v; // Exponent public (65537)
    private int k; // Dimensiune challenge (256 biti)

    // Parametri SECRETI - doar CA
    private BigInteger p, q; // Factori primi
    private BigInteger s; // Exponent privat:  $s = v^{-1} \bmod \phi(n)$ 

    // Utilitare
    private SecureRandom random;
    private MessageDigest hash;

    /**
     * Constructor pentru Autoritatea de Certificare (CA)
     * Genereaza toti parametrii sistemului
     */
    public GQSignature(int bitLength, int k) throws Exception {
        this.k = k;
        this.random = new SecureRandom();
        this.hash = MessageDigest.getInstance("SHA-256");
        genereareParametri(bitLength);
    }

    /**
     * Constructor pentru Utilizatori
     * Foloseste parametri publici existenti
     */
    public GQSignature(BigInteger n, BigInteger v, int k) throws Exception {
        this.n = n;
        this.v = v;
```

```

    this.k = k;
    this.random = new SecureRandom();
    this.hash = MessageDigest.getInstance("SHA-256");
}

/**
 * PASUL 1: Generare parametri sistem (doar CA)
 * Genereaza: p, q, n, v, s
 */
private void genereareParametri(int bitLength) {
    // Genereaza doi factori primi mari
    p = BigInteger.probablePrime(bitLength / 2, random);
    q = BigInteger.probablePrime(bitLength / 2, random);

    // Calculeaza  $n = p \times q$ 
    n = p.multiply(q);

    // Calculeaza  $\phi(n) = (p-1)(q-1)$ 
    BigInteger phi = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));

    // Alege  $v = 65537$  (standard RSA)
    v = BigInteger.valueOf(65537);

    // Calculeaza  $s = v^{-1} \bmod \phi(n)$ 
    s = v.modInverse(phi);
}

/**
 * PASUL 2: Generare certificat pentru utilizator (doar CA)
 * Input: identitate (ex: "alice@company.com")
 * Output: J = certificat secret
 */
public BigInteger genereareCertificat(String identitate) {
    //  $I = H(\text{identitate}) \bmod n$ 
    BigInteger I = hashCatreBigInteger(identitate).mod(n);

    //  $J = I^s \bmod n$  (certificatul secret)
    BigInteger J = I.modPow(s, n);

    return J;
}

/**
 * PASUL 3: Semnare mesaj
 * Input: mesaj, identitate, certificat
 * Output: semnatura (d, y, identitate)
 */
public Semnatura semnare(String mesaj, String identitate, BigInteger certificat) {
    // 1. Alege  $r$  aleatoriu din  $Z_n^*$ 
    BigInteger r = generareRandom();

```

```

// 2. Calculeaza commitment:  $T = r^v \bmod n$ 
BigInteger T = r.modPow(v, n);

// 3. Calculeaza challenge:  $d = H(\text{mesaj} \parallel T) \bmod 2^k$ 
BigInteger d = calculeazaChallenge(mesaj, T);

// 4. Calculeaza raspuns:  $y = r \times J^d \bmod n$ 
BigInteger Jd = certificat.modPow(d, n);
BigInteger y = r.multiply(Jd).mod(n);

return new Semnatura(d, y, identitate);
}

/**
 * PASUL 4: Verificare semnatura
 * Input: mesaj, semnatura
 * Output: true daca valida, false altfel
 */
public boolean verificare(String mesaj, Semnatura sem) {
    try {
        // 1. Recalculeaza  $I = H(\text{identitate}) \bmod n$ 
        BigInteger I = hashCatreBigInteger(sem.identitate).mod(n);

        // 2. Calculeaza  $T' = y^v \times I^{-d} \bmod n$ 
        BigInteger yv = sem.y.modPow(v, n);
        BigInteger Id = I.modPow(sem.d, n);
        BigInteger IdInv = Id.modInverse(n);
        BigInteger TPrim = yv.multiply(IdInv).mod(n);

        // 3. Recalculeaza challenge:  $d' = H(\text{mesaj} \parallel T')$ 
        BigInteger dPrim = calculeazaChallenge(mesaj, TPrim);

        // 4. Verifica:  $d' == d$ 
        return dPrim.equals(sem.d);

    } catch (Exception e) {
        return false;
    }
}

// ===== FUNCTII AUXILIARE =====

/**
 * Genereaza numar aleatoriu r din  $\mathbb{Z}_n^*$ 
 * Conditii:  $r < n$  si  $\gcd(r, n) = 1$ 
 */
private BigInteger generareRandom() {
    BigInteger r;
    do {

```

```

        r = new BigInteger(n.bitLength(), random);
    } while (r.compareTo(n) >= 0 || !r.gcd(n).equals(BigInteger.ONE));
    return r;
}

/**
 * Hash string catre BigInteger
 */
private BigInteger hashCatreBigInteger(String text) {
    hash.reset();
    byte[] hashBytes = hash.digest(text.getBytes());
    return new BigInteger(1, hashBytes);
}

/**
 * Calculeaza challenge:  $d = H(\text{mesaj} \parallel T) \bmod 2^k$ 
 */
private BigInteger calculeazaChallenge(String mesaj, BigInteger T) {
    hash.reset();
    String combinat = mesaj + T.toString();
    byte[] hashBytes = hash.digest(combinat.getBytes());
    BigInteger hashInt = new BigInteger(1, hashBytes);

    // Reduce la k biti: hash mod  $2^k$ 
    BigInteger modulus = BigInteger.ONE.shiftLeft(k);
    return hashInt.mod(modulus);
}

// Getteri pentru parametri publici
public BigInteger getN() { return n; }
public BigInteger getV() { return v; }
public int getK() { return k; }

/**
 * Clasa pentru stocare semnatura
 */
public static class Semnatura {
    public final BigInteger d; // Challenge
    public final BigInteger y; // Raspuns
    public final String identitate;

    public Semnatura(BigInteger d, BigInteger y, String identitate) {
        this.d = d;
        this.y = y;
        this.identitate = identitate;
    }
}

@Override
public String toString() {
    return String.format("Semnatura GQ [d=%d biti, y=%d biti, ID=%s]",

```

```

        d.bitLength(), y.bitLength(), identitate);
    }
}
}

```

2. GQDemo.java - Demonstrație completă cu doi utilizatori (Alice și Bob) Include teste de integritate și autenticitate

```

import java.math.BigInteger;

/**
 * Demonstratie functionare schema Guillou-Quisquater
 * Exemplu complet cu 2 utilizatori (Alice si Bob)
 */
public class GQDemo {

    public static void main(String[] args) {
        try {
            afiseazaSeparator();
            System.out.println("DEMONSTRATIE SCHEMA GUILLOU-QUISQUATER");
            afiseazaSeparator();

            // ===== PASUL 1: SETUP SISTEM =====
            System.out.println("\n[PASUL 1] SETUP SISTEM - Autoritatea de Certificare (CA)");
            System.out.println("-".repeat(70));

            int dimensiuneCheie = 1024; // biti
            int parametruSecuritate = 256; // k = 256 biti (SHA-256)

            System.out.println("CA genereaza parametri sistem...");
            GQSignature autoritate = new GQSignature(dimensiuneCheie, parametruSecuritate);

            System.out.println("Parametri PUBLICI generati:");
            System.out.println("  n (modul):          " + autoritate.getN().bitLength() + " biti");
            System.out.println("  Valoare (primii 40 hex): " + autoritate.getN().toString(16).substring(0, 40) + "...");
            System.out.println("  v (exponent public):  " + autoritate.getV());
            System.out.println("  k (parametru securitate): " + autoritate.getK() + " biti");

            // ===== PASUL 2: INREGISTRARE UTILIZATORI =====
            System.out.println("\n[PASUL 2] INREGISTRARE UTILIZATORI");
            System.out.println("-".repeat(70));

            String idAlice = "alice@company.com";
            String idBob = "bob@company.com";

            System.out.println("Generare certificate pentru utilizatori...");

            BigInteger certAlice = autoritate.generareCertificat(idAlice);
            System.out.println("  Alice (" + idAlice + ")");
            System.out.println("  Certificat: " + certAlice.bitLength() + " biti");

```



```

System.out.println("  Valoare (primii 40 hex): " + certAlice.toString(16).substring(0, 40) + "...");

BigInteger certBob = autoritate.generareCertificat(idBob);
System.out.println("  Bob (" + idBob + ")");
System.out.println("  Certificat: " + certBob.bitLength() + " biti");
System.out.println("  Valoare (primii 40 hex): " + certBob.toString(16).substring(0, 40) + "...");

// ===== PASUL 3: UTILIZATORII PRIMESC PARAMETRI =====
System.out.println("\n[PASUL 3] DISTRIBUIRE PARAMETRI PUBLICI");
System.out.println("-".repeat(70));

GQSignature alice = new GQSignature(autoritate.getN(), autoritate.getV(), autoritate.getK());
GQSignature bob = new GQSignature(autoritate.getN(), autoritate.getV(), autoritate.getK());

System.out.println("Alice si Bob au primit parametrii publici (n, v, k)");
System.out.println("Acum pot semna mesaje si verifica semnaturi!");

// ===== PASUL 4: ALICE SEMNEAZA =====
System.out.println("\n[PASUL 4] ALICE SEMNEAZA UN MESAJ");
System.out.println("-".repeat(70));

String mesaj1 = "Contract: Transfer 1000 EUR catre Bob";
System.out.println("Mesaj: \"" + mesaj1 + "\"");
System.out.println("\nAlice semneaza...");

GQSignature.Semnatura semAlice = alice.semna(mesaj1, idAlice, certAlice);

System.out.println("Semnatura generata:");
System.out.println("  Challenge (d): " + semAlice.d.bitLength() + " biti");
System.out.println("  Valoare (hex): " + semAlice.d.toString(16));
System.out.println("  Raspuns (y): " + semAlice.y.bitLength() + " biti");
System.out.println("  Valoare (primii 40 hex): " + semAlice.y.toString(16).substring(0, 40) + "...");
System.out.println("  Identitate: " + semAlice.identitate);

// ===== PASUL 5: BOB VERIFICA =====
System.out.println("\n[PASUL 5] BOB VERIFICA SEMNATURA LUI ALICE");
System.out.println("-".repeat(70));

boolean valid1 = bob.verificare(mesaj1, semAlice);
System.out.println("Rezultat verificare: " + (valid1 ? "VALIDA" : "INVALIDA"));

if (!valid1) {
    System.out.println("EROARE: Semnatura ar trebui sa fie valida!");
    return;
}

// ===== PASUL 6: TEST INTEGRITATE =====
System.out.println("\n[PASUL 6] TEST INTEGRITATE - Mesaj Modificat");
System.out.println("-".repeat(70));

```

```
String mesajModificat = "Contract: Transfer 9000 EUR catre Bob"; // MODIFICAT!  
System.out.println("Mesaj modificat: \"\" + mesajModificat + \"\"");
```

```
boolean valid2 = bob.verificare(mesajModificat, semAlice);  
System.out.println("Rezultat verificare: " + (valid2 ? "VALIDA" : "INVALIDA"));
```

```
if (valid2) {  
    System.out.println("EROARE: Mesajul modificat ar trebui sa fie invalid!");  
    return;  
}  
System.out.println("Corect! Mesajul modificat a fost detectat.");
```

```
// ===== PASUL 7: TEST AUTENTICITATE =====  
System.out.println("\n[PASUL 7] TEST AUTENTICITATE - Identitate Falsa");  
System.out.println("-".repeat(70));
```

```
GQSignature.Semnatura semFalsa =  
    new GQSignature.Semnatura(semAlice.d, semAlice.y, idBob); // Identitate falsa!
```

```
System.out.println("Incercare: semnatura lui Alice cu identitatea lui Bob");  
boolean valid3 = bob.verificare(mesaj1, semFalsa);  
System.out.println("Rezultat verificare: " + (valid3 ? "VALIDA" : "INVALIDA"));
```

```
if (valid3) {  
    System.out.println("EROARE: Identitatea falsa ar trebui sa fie invalida!");  
    return;  
}  
System.out.println("Corect! Identitatea falsa a fost detectata.");
```

```
// ===== PASUL 8: BOB SEMNEAZA =====  
System.out.println("\n[PASUL 8] BOB SEMNEAZA UN MESAJ");  
System.out.println("-".repeat(70));
```

```
String mesaj2 = "Confirm primirea sumei de 1000 EUR de la Alice";  
System.out.println("Mesaj: \"\" + mesaj2 + \"\"");  
System.out.println("\nBob semneaza...");
```

```
GQSignature.Semnatura semBob = bob.semnare(mesaj2, idBob, certBob);
```

```
System.out.println("Semnatura generata de Bob:");  
System.out.println(" Challenge (d): " + semBob.d.bitLength() + " biti");  
System.out.println(" Valoare (hex): " + semBob.d.toString(16));  
System.out.println(" Raspuns (y): " + semBob.y.bitLength() + " biti");  
System.out.println(" Valoare (primii 40 hex): " + semBob.y.toString(16).substring(0, 40) + "...");
```

```
// ===== PASUL 9: ALICE VERIFICA =====  
System.out.println("\n[PASUL 9] ALICE VERIFICA SEMNATURA LUI BOB");  
System.out.println("-".repeat(70));
```

```
boolean valid4 = alice.verificare(mesaj2, semBob);
```

```

System.out.println("Rezultat verificare: " + (valid4 ? "VALIDA" : "INVALIDA"));

if (!valid4) {
    System.out.println("EROARE: Semnatura lui Bob ar trebui sa fie valida!");
    return;
}

// ===== STATISTICI FINALE =====
afiseazaSeparator();
System.out.println("STATISTICI FINALE");
afiseazaSeparator();

int dimensiuneSemnatura = (semAlice.d.bitLength() + semAlice.y.bitLength()) / 8;

System.out.println("Dimensiune modul (n):  " + dimensiuneCheie + " biti");
System.out.println("Dimensiune certificat:  " + certAlice.bitLength() + " biti");
System.out.println("Dimensiune semnatura:  ~" + dimensiuneSemnatura + " bytes");
System.out.println("Parametru securitate (k): " + parametruSecuritate + " biti");

afiseazaSeparator();
System.out.println("TOATE TESTELE AU FOST EXECUTATE CU SUCCES!");
afiseazaSeparator();

} catch (Exception e) {
    System.err.println("\nEROARE: " + e.getMessage());
    e.printStackTrace();
}
}

private static void afiseazaSeparator() {
    System.out.println("\n" + "=".repeat(70));
}
}

```

3. GQBenchmark.java - Măsurători comparative de performanță între algoritmi GQ și RSA Sunt testate chei cu dimensiuni de 1024 și 2048 de biți

```

import java.math.BigInteger;
import java.security.*;
import javax.crypto.Cipher;

/**
 * Benchmark comparativ: GQ vs RSA
 * Masoara: setup, semnare, verificare, dimensiuni
 */
public class GQBenchmark {

    private static final int ITERATII = 100;
    private static final String MESAJ_TEST = "Acesta este un mesaj de test pentru semnatura digitala";
    private static final String IDENTITATE_TEST = "user@example.com";

```

```

public static void main(String[] args) {
    afiseazaSeparator();
    System.out.println("BENCHMARK GUILLOU-QUISQUATER vs RSA");
    afiseazaSeparator();

    try {
        // Testeaza diferite dimensiuni de chei
        int[] dimensiuni = {1024, 2048};

        for (int dim : dimensiuni) {
            System.out.println("\n--- Dimensiune cheie: " + dim + " biti ---\n");

            benchmarkGQ(dim);
            System.out.println();

            benchmarkRSASignature(dim);
            System.out.println();

            benchmarkRSAEncryption(dim);
            System.out.println();

            afiseazaSeparator();
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}

/**
 * Benchmark pentru schema GQ
 */
private static void benchmarkGQ(int dimensiuneCheie) throws Exception {
    System.out.println("GUILLOU-QUISQUATER:");

    // 1. SETUP sistem
    long timpStart = System.nanoTime();
    GQSignature autoritate = new GQSignature(dimensiuneCheie, 256); // k=256
    long timpSetup = (System.nanoTime() - timpStart) / 1_000_000;

    // 2. GENERARE certificat
    timpStart = System.nanoTime();
    BigInteger certificat = autoritate.generareCertificat(IDENTITATE_TEST);
    long timpCertificat = (System.nanoTime() - timpStart) / 1_000_000;

    // 3. Creeare instanta utilizator
    GQSignature utilizator = new GQSignature(
        autoritate.getN(),
        autoritate.getV(),

```

```

    autoritate.getK()
);

// 4. WARMUP (incalzire JVM)
for (int i = 0; i < 10; i++) {
    GQSignature.Semnatura sem = utilizator.semnare(MESAJ_TEST, IDENTITATE_TEST, certificat);
    utilizator.verificare(MESAJ_TEST, sem);
}

// 5. BENCHMARK semnare
timpStart = System.nanoTime();
GQSignature.Semnatura semnatura = null;
for (int i = 0; i < ITERATII; i++) {
    semnatura = utilizator.semnare(MESAJ_TEST, IDENTITATE_TEST, certificat);
}
long timpSemnare = (System.nanoTime() - timpStart) / ITERATII / 1_000_000;

// 6. BENCHMARK verificare
timpStart = System.nanoTime();
boolean valid = false;
for (int i = 0; i < ITERATII; i++) {
    valid = utilizator.verificare(MESAJ_TEST, semnatura);
}
long timpVerificare = (System.nanoTime() - timpStart) / ITERATII / 1_000_000;

// 7. AFISARE rezultate
System.out.printf(" Setup sistem:      %6d ms\n", timpSetup);
System.out.printf(" Generare certificat:  %6d ms\n", timpCertificat);
System.out.printf(" Semnare (medie):      %6d ms\n", timpSemnare);
System.out.printf(" Verificare (medie):   %6d ms\n", timpVerificare);
System.out.printf(" Verificare valida:    %s\n", valid ? "DA" : "NU");

int dimSemnatura = (semnatura.d.bitLength() + semnatura.y.bitLength()) / 8;
System.out.printf(" Dimensiune semnatura:  %d bytes\n", dimSemnatura);
}

/**
 * Benchmark pentru RSA Signature
 */
private static void benchmarkRSASignature(int dimensiuneCheie) throws Exception {
    System.out.println("RSA SIGNATURE (SHA256withRSA):");

    // 1. SETUP (generare chei)
    long timpStart = System.nanoTime();
    KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
    keyGen.initialize(dimensiuneCheie, new SecureRandom());
    KeyPair keyPair = keyGen.generateKeyPair();
    long timpSetup = (System.nanoTime() - timpStart) / 1_000_000;

    Signature rsaSign = Signature.getInstance("SHA256withRSA");

```

```

byte[] mesajBytes = MESAJ_TEST.getBytes();

// 2. WARMUP
for (int i = 0; i < 10; i++) {
    rsaSign.initSign(keyPair.getPrivate());
    rsaSign.update(mesajBytes);
    byte[] sem = rsaSign.sign();

    rsaSign.initVerify(keyPair.getPublic());
    rsaSign.update(mesajBytes);
    rsaSign.verify(sem);
}

// 3. BENCHMARK semnare
timpStart = System.nanoTime();
byte[] semnatura = null;
for (int i = 0; i < ITERATII; i++) {
    rsaSign.initSign(keyPair.getPrivate());
    rsaSign.update(mesajBytes);
    semnatura = rsaSign.sign();
}
long timpSemnare = (System.nanoTime() - timpStart) / ITERATII / 1_000_000;

// 4. BENCHMARK verificare
timpStart = System.nanoTime();
boolean valid = false;
for (int i = 0; i < ITERATII; i++) {
    rsaSign.initVerify(keyPair.getPublic());
    rsaSign.update(mesajBytes);
    valid = rsaSign.verify(semnatura);
}
long timpVerificare = (System.nanoTime() - timpStart) / ITERATII / 1_000_000;

// 5. AFISARE rezultate
System.out.printf(" Generare chei:      %6d ms\n", timpSetup);
System.out.printf(" Semnare (medie):      %6d ms\n", timpSemnare);
System.out.printf(" Verificare (medie):   %6d ms\n", timpVerificare);
System.out.printf(" Verificare valida:   %s\n", valid ? "DA" : "NU");
System.out.printf(" Dimensiune semnatura: %d bytes\n", semnatura.length);
}

/**
 * Benchmark pentru RSA Encryption/Decryption
 */
private static void benchmarkRSAEncryption(int dimensiuneCheie) throws Exception {
    System.out.println("RSA ENCRYPTION/DECRYPTION (PKCS1Padding):");

    // 1. SETUP (generare chei)
    long timpStart = System.nanoTime();
    KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");

```

```

keyGen.initialize(dimensiuneCheie, new SecureRandom());
KeyPair keyPair = keyGen.generateKeyPair();
long timpSetup = (System.nanoTime() - timpStart) / 1_000_000;

Cipher cipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");

// Mesaj scurt pentru criptare (limitare RSA)
byte[] mesajScurt = "Test".getBytes();

// 2. WARMUP
for (int i = 0; i < 10; i++) {
    cipher.init(Cipher.ENCRYPT_MODE, keyPair.getPublic());
    byte[] criptat = cipher.doFinal(mesajScurt);
    cipher.init(Cipher.DECRYPT_MODE, keyPair.getPrivate());
    cipher.doFinal(criptat);
}

// 3. BENCHMARK criptare
timpStart = System.nanoTime();
byte[] criptat = null;
for (int i = 0; i < ITERATII; i++) {
    cipher.init(Cipher.ENCRYPT_MODE, keyPair.getPublic());
    criptat = cipher.doFinal(mesajScurt);
}
long timpCriptare = (System.nanoTime() - timpStart) / ITERATII / 1_000_000;

// 4. BENCHMARK decriptare
timpStart = System.nanoTime();
for (int i = 0; i < ITERATII; i++) {
    cipher.init(Cipher.DECRYPT_MODE, keyPair.getPrivate());
    cipher.doFinal(criptat);
}
long timpDecriptare = (System.nanoTime() - timpStart) / ITERATII / 1_000_000;

// 5. AFISARE rezultate
System.out.printf(" Generare chei:      %6d ms\n", timpSetup);
System.out.printf(" Criptare (medie):    %6d ms\n", timpCriptare);
System.out.printf(" Decriptare (medie):  %6d ms\n", timpDecriptare);
System.out.printf(" Dimensiune criptat:  %d bytes\n", criptat.length);
}

private static void afiseazaSeparator() {
    System.out.println("=".repeat(70));
}
}

```